설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

문자열 연산과 검색

posco

설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

문자열의 이해와 생성 Part 1

문자열 만들기

자료형과 형변환

인덱싱과 슬라이싱 Part 2

인덱싱

슬라이싱

연산·검색과 정리 메서드 Part 3

정리 메서드

f-string과 텍스트 정리 Part 4

f-string

문자열 연산과 검색

posco

여러 변수 문자열 연결

더하기를 여러 번 이어 여러 문자열을 한 줄로

code + '/' + status + '/' + date 형태

사이에 넣는 '/', '-'를 **구분자**라 함

구분자에 따라 결과 모양이 달라짐

구분선 출력 - 코드 예시

반복(*)으로 구분선, 연결(+)로 라벨·값

CODE

```
status = '정상'
line = '=' * 20
print(line)
print('상태: ' + status)
print('횟수: ' + str(5))
```

len() - 문자열 길이 구하기

len() - 문자열의 글자 수를 숫자로 알려 줌

len('PYTHON')은 6

공백도 한 글자로 셈 ('AI 분석'은 5)

빈 문자열의 길이는 0

len() 활용 - 입력값 길이 검사

len()으로 입력값이 **규칙에 맞는지** 점검

휴대폰 번호 11자리, 설비 코드 7자리 확인

지금은 len 결과를 print로 눈으로 확인

자동 판단은 조건문 학습 후

실습 1. 길이 재기

목표	len() 으로 문자열의 길이(자릿수)를 확인
단계	① 전화번호 '01012345678' 을 변수에 저장 → ② len() 으로 자릿수 출력
예상 결과	11

in - 포함 여부 확인

in - 특정 단어가 들어 있는지 참·거짓으로 확인

'고장' in status가 들어 있으면 True

없으면 False

결과는 True 또는 False 두 가지 (불리언)

not in - 미포함 확인

Q1. not in은 무엇인가요?

A. 들어 있지 않으면 True, 있으면 False (in의 반대)

Q2. in과 어떤 관계인가요?

A. 같은 상황에서 항상 반대 결과

Q3. 어느 쪽을 쓰나요?

A. 문장으로 읽어 자연스러운 쪽

in 활용 - 키워드·상태 탐색

상태·로그에서 **위험 키워드**를 in으로 탐색

'WARNING' in message로 경고 포함 확인

컬럼명에 'sensor' 포함 여부도 in으로

다음 단원 데이터 필터링의 기초 개념

실습 2. 포함 확인하기

목표	in 으로 키워드가 문자열에 들어있는지 확인
단계	① 메시지 '설비 고장 발생' 저장 → ② '고장'·'정상' 이 in 인지 각각 출력
예상 결과	True / False

count() - 개수 세기

count() - 특정 글자·단어가 몇 번 나오는지 셈

'banana'.count('a')는 3

'a,b,c,d'.count(',')는 3 (항목 4개)

없으면 0 (오류 아닌 정상 결과)

실습 3. 개수 세기

목표	count() 로 특정 글자가 몇 번 나오는지 세기
단계	① 'sensor-01-temp' 저장 → ② count('-') 로 하이픈 개수 출력
예상 결과	2

find() - 위치 찾기

find() - 특정 글자가 처음 나오는 위치(번호)

'hong@company.com'.find('@')는 4

찾은 위치를 슬라이싱 끝 번호로 활용

없으면 -1 (못 찾았다는 신호)

find - 코드 예시

find로 위치를 찾아 슬라이싱과 연결

CODE

```
email = 'hong@company.com'
at = email.find('@')
print(at)              # 4
print(email[:at])      # hong
print('정상'.find('고장'))   # -1
```

실습 4. 위치 찾기

목표	find() 로 특정 글자가 처음 나오는 위치(번호) 찾기
단계	① 'temp_2024' 저장 → ② find('_') 로 첫 위치, 없는 글자는 -1 확인
예상 결과	4 / -1

index() - 위치 찾기 (find과 짝)

index() - 위치를 반환, 사용법은 find과 동일 (단, 없으면 오류)

'hong@company.com'.index('@')는 4 (find과 같은 결과)

차이 - 없을 때 find는 -1, index는 오류로 멈춤

없어도 멈추면 안 되면 find / 반드시 있는 값이면 index

startswith() - 시작 확인

startswith() - 특정 단어로 시작하는지 참·거짓

'EQP-001'.startswith('EQP')는 True

in과 차이 - in은 어디든 포함, startswith는 맨 앞

'data_log'에서 'log'는 in True, startswith False

endswith() - 끝 확인

endswith() - 특정 단어로 끝나는지 참·거짓

'cmapss.csv'.endswith('.csv')는 True

파일 확장자 확인의 단골 도구

startswith는 앞, endswith는 뒤 (짝으로 기억)

실습 5. 시작과 끝 확인하기

목표	파일명이 정해진 접두어로 시작·확장자로 끝나는지 판별
단계	① 'sensor_log.csv' 저장 → ② startswith('sensor')·endswith('.csv') 확인
예상 결과	True / True

== 로 문자열 비교

== (등호 두 개)로 두 문자열이 같은지 비교

status == '정상'이 같으면 True

등호 하나는 저장, 두 개는 비교

'정상'과 '정상 '(뒤 공백)은 다른 값

대소문자 다르면 다른 문자열

컴퓨터는 대문자와 소문자를 **다른 글자**로 봄

'NORMAL' == 'normal'은 False

같은 상태가 'NORMAL' 'Normal'로 섞이면 따로 집계

비교 전 대소문자 통일 필요 (다음 모듈)

실습 6. 대소문자 구분 비교하기

목표	== 비교가 대소문자를 구분함을 확인
단계	① 상태 라벨 'WARNING' 저장 → ② 'WARNING'·'warning' 과 각각 == 비교
예상 결과	True / False

연산·검색 메서드 정리

도구를 **결과 형태**로 묶어 기억

숫자를 주는 것 - len, count, find

참·거짓을 주는 것 - in, startswith, ==

없을 때 - count는 0, find는 -1

연산·검색 도구 정리표

도구	용도	결과
+ / *	연결·반복	문자열
len / count / find	길이·횟수·위치	숫자 (없으면 0/-1)
in / not in	포함 여부	참·거짓
startswith / endswith / ==	시작·끝·비교	참·거짓

활용 - 컬럼명·파일명 점검

검색 도구를 묶어 **이름표**가 규칙에 맞는지 점검

endswith로 csv 확인, in으로 키워드 확인

startswith로 접두어, len으로 길이, find로 위치

여러 도구를 목적에 맞게 골라 쓰기

감사합니다.

posco